PRAKTIKUM 6

MENGEMBANGKAN FUNGSI-FUNGSI SENDIRI

Pemrograman C++

Nama	**.....**
NRP	.....
Kelas	.....
Tanggal Praktikum	.....

6.1 Judul

Mengembangkan Fungsi-Fungsi Sendiri dalam C++

6.2 Tujuan Praktikum

- Memahami fungsi sebagai bagian program yang menjalankan tugas tertentu.
- Membedakan prototipe, definisi, pemanggilan fungsi, parameter, dan argumen.
- Membuat fungsi void dan fungsi yang mengembalikan nilai.
- Menggunakan fungsi dengan parameter serta nilai balik.
- Memahami variabel lokal, global, otomatis, dan lokal static.
- Menguji program dengan kasus normal, batas, dan masukan tidak valid.

6.3 Dasar Teori

A. Pengertian fungsi

Fungsi adalah bagian program bernama yang dirancang untuk menjalankan tugas tertentu. Fungsi memecah masalah menjadi bagian yang lebih kecil, mengurangi pengulangan kode, dan memudahkan pengujian serta pemeliharaan program.

B. Prototipe, definisi, dan pemanggilan

Prototipe menyatakan antarmuka fungsi agar compiler mengetahui tipe nilai balik dan parameternya. Definisi berisi perintah yang dijalankan. Pemanggilan fungsi menjalankan fungsi tersebut.

```
int tambah(int a, int b);  // prototipe

int tambah(int a, int b) {   // definisi
    return a + b;
}

int hasil = tambah(3, 4);    // pemanggilan
```

C. Parameter, argumen, dan nilai balik

Parameter adalah variabel dalam daftar parameter fungsi, sedangkan argumen adalah nilai yang dikirim saat fungsi dipanggil. Fungsi bertipe void tidak mengembalikan nilai. Fungsi bertipe int, double, bool, atau tipe lain dapat mengembalikan nilai menggunakan return.

```
void tampilkanPesan() {
    std::cout << "Selamat belajar fungsi!\n";
}

int kuadrat(int n) {
    return n * n;
}
```

D. Variabel lokal, global, dan static

Variabel lokal hanya dapat diakses pada ruang lingkup tempat variabel dideklarasikan. Variabel global dideklarasikan di luar fungsi. Variabel lokal static mempertahankan nilainya di antara pemanggilan fungsi. Penggunaan variabel global perlu dibatasi agar aliran data program mudah ditelusuri.

```
int jumlahPemanggilan = 0; // global

void demo() {
    int otomatis = 0;       // dibuat ulang setiap pemanggilan
    static int tetap = 0;   // mempertahankan nilai
    ++otomatis;
    ++tetap;
    ++jumlahPemanggilan;
    std::cout << otomatis << " " << tetap << " "
              << jumlahPemanggilan << '\n';
}
```

Catatan: contoh lama dapat memakai void main(), conio.h, clrscr(), getch(), dan register. Praktikum revisi menggunakan int main(), iostream, serta return 0. Kata kunci register tidak diperlukan dalam C++ modern.

6.4 Program Percobaan

Percobaan 1 — Fungsi tanpa parameter dan tanpa nilai balik

Buat program yang memanggil tampilkanPesan() sebanyak lima kali. Perhatikan prototipe sebelum main() dan definisi setelah main().

```
#include <iostream>

void tampilkanPesan();

int main() {
    for (int i = 1; i <= 5; ++i) {
        std::cout << "Pesan ke-" << i << ": ";
        tampilkanPesan();
    }
    return 0;
}

void tampilkanPesan() {
    std::cout << "Belajar fungsi C++\n";
}
```

Percobaan 2 — Fungsi dengan parameter

Fungsi berikut menghitung jumlah bilangan dari 1 sampai n. Jalankan untuk n = 10, 20, dan 50, lalu jelaskan hubungan argumen dengan parameter.

```
#include <iostream>

int jumlahSampai(int n) {
    int jumlah = 0;
    for (int i = 1; i <= n; ++i) {
        jumlah += i;
    }
    return jumlah;
}

int main() {
    std::cout << jumlahSampai(10) << '\n';
    std::cout << jumlahSampai(20) << '\n';
    std::cout << jumlahSampai(50) << '\n';
    return 0;
}
```

Percobaan 3 — Fungsi FPB dengan nilai balik

Implementasikan algoritma Euclid untuk menghitung Faktor Persekutuan Terbesar (FPB). Fungsi menerima dua bilangan bulat positif dan mengembalikan FPB.

```
#include <iostream>

int fpb(int a, int b) {
    while (b != 0) {
        int sisa = a % b;
        a = b;
        b = sisa;
    }
    return a;
}

int main() {
    std::cout << "FPB(150, 35) = " << fpb(150, 35) << '\n';
    std::cout << "FPB(1026, 405) = " << fpb(1026, 405) << '\n';
    std::cout << "FPB(83, 240) = " << fpb(83, 240) << '\n';
    return 0;
}
```

Percobaan 4 — Fungsi memanggil fungsi lain

Lengkapi fungsi nilaiAbsolut() dan akarKuadrat() menggunakan metode Newton. Fungsi akarKuadrat() harus menangani masukan negatif dan nol. Nilai -1.0 digunakan sebagai penanda masukan negatif pada contoh ini.

```
#include <iostream>

double nilaiAbsolut(double x) {
    return (x < 0) ? -x : x;
}

double akarKuadrat(double x) {
    if (x < 0) return -1.0;
    if (x == 0) return 0.0;

    double tebakan = x;
    const double epsilon = 0.00001;
    while (nilaiAbsolut(tebakan * tebakan - x) >= epsilon) {
        tebakan = (tebakan + x / tebakan) / 2.0;
    }
    return tebakan;
}

int main() {
    std::cout << akarKuadrat(4.0) << '\n';
    std::cout << akarKuadrat(625.0) << '\n';
    std::cout << akarKuadrat(-39.5) << '\n';
    return 0;
}
```

Percobaan 5 — Fungsi nilai minimum

Jalankan fungsi minimum() untuk membandingkan dua bilangan double. Uji bilangan positif, negatif, dan dua bilangan yang sama.

```
#include <iostream>

double minimum(double x, double y) {
    if (x < y) return x;
    return y;
}

int main() {
    double a, b;
    std::cout << "Masukkan dua bilangan: ";
    if (!(std::cin >> a >> b)) {
        std::cout << "Masukan tidak valid\n";
```

```
        return 1;
    }
    std::cout << "Nilai minimum = " << minimum(a, b) << '\n';
    return 0;
}
```

Percobaan 6 — Variabel otomatis dan static
Panggil demo() tiga kali. Catat perbedaan nilai variabel otomatis dan static pada setiap pemanggilan.

```
#include <iostream>

void demo() {
    int otomatis = 0;
    static int tetap = 0;
    ++otomatis;
    ++tetap;
    std::cout << "otomatis=" << otomatis
              << ", static=" << tetap << '\n';
}

int main() {
    for (int i = 0; i < 3; ++i) demo();
    return 0;
}
```

6.5 Tugas Praktikum

Kerjakan tugas berikut secara mandiri. Setiap program harus memiliki nama berkas .cpp, komentar singkat pada algoritma penting, minimal tiga pengujian, dan bukti hasil eksekusi.

6.5.1 Fungsi kabisat — Buat bool kabisat(int tahun) yang mengembalikan true jika tahun kabisat dan false jika bukan. Gunakan aturan kalender Gregorian.

6.5.2 Faktorial — Buat fungsi faktorial(int n) untuk menghitung n! bagi n ≥ 0. Validasi masukan negatif dan jelaskan batas tipe data yang digunakan.

6.5.3 Permutasi dan kombinasi — Buat fungsi permutasi(n, r) dan kombinasi(n, r) berdasarkan P(n,r) = n!/(n-r)! dan C(n,r) = n!/[r!(n-r)!]. Validasi 0 ≤ r ≤ n.

6.5.4 Konversi suhu — Buat fungsi konversiSuhu(suhu, asal, tujuan) untuk konversi Celsius, Fahrenheit, dan Réaumur. Tampilkan hasil dengan dua angka desimal dan jelaskan rumusnya.

6.5.5 Bilangan prima — Buat bool prima(int n). Uji nilai kurang dari 2, bilangan prima, dan bilangan komposit.

6.5.6 Variabel otomatis dan static — Buat dua fungsi yang masing-masing menggunakan variabel otomatis dan static. Panggil setiap fungsi lima kali dan jelaskan perbedaan keluarannya.

6.5.7 Nilai terbesar — Buat fungsi terbesarTiga(a, b, c) untuk mengembalikan bilangan terbesar dari tiga bilangan tanpa menggunakan array.

6.5.8 Validasi segitiga — Buat fungsi validSegitiga(a, b, c) yang memeriksa sisi positif dan ketaksamaan segitiga. Tentukan pula jenis segitiganya.

6.5.9 Rekursi — Buat fungsi rekursif jumlahDigit(n) untuk menghitung jumlah digit bilangan bulat nonnegatif. Bandingkan dengan solusi iteratif.

6.5.10 Analisis digit — Buat fungsi hitungDigit(n), balikDigit(n), dan palindrom(n). Uji nol, bilangan satu digit, dan digit berulang.

10 soal analisis

1. Apa perbedaan prototipe fungsi dan definisi fungsi?

2. Apa perbedaan parameter dan argumen?

3. Mengapa fungsi yang mengembalikan nilai menggunakan return?

4. Apa risiko jika fungsi bertipe int tidak mengembalikan nilai pada jalur eksekusi yang seharusnya?

5. Jelaskan perbedaan variabel lokal, global, dan lokal static.

6. Mengapa penggunaan variabel global secara berlebihan menyulitkan pemeliharaan program?

7. Apa keuntungan membagi program menjadi beberapa fungsi dibanding menulis seluruh instruksi di main()?

8. Mengapa fungsi akar kuadrat perlu menangani masukan negatif?

9. Bandingkan pendekatan iteratif dan rekursif dari kondisi berhenti serta penggunaan sumber daya.

10. Pilih satu tugas, lalu jelaskan tiga kasus uji: normal, batas, dan tidak valid, beserta keluaran yang diharapkan.

6.6 Format Pengumpulan dan Penilaian

- Kode sumber (.cpp) untuk setiap percobaan dan tugas.
- Bukti eksekusi yang terbaca, termasuk masukan dan keluaran.
- Jawaban 10 soal analisis serta penjelasan singkat algoritma.
- Gunakan nama variabel yang jelas, indentasi konsisten, dan kode yang mudah dibaca.

Rubrik penilaian yang disarankan: ketepatan program 40%; penerapan konsep fungsi 25%; pengujian dan validasi 15%; analisis 10%; kerapian dan dokumentasi 10%.

Checklist Sebelum Dikumpulkan

- Semua program dapat dikompilasi tanpa error.
- Setiap fungsi memiliki tujuan yang jelas.
- Parameter dan tipe nilai balik sesuai kebutuhan.
- Kasus normal, batas, dan masukan tidak valid telah diuji.
- Kode sumber, bukti eksekusi, dan jawaban analisis sudah disertakan.